

Code Explained

how my project actually works

Every important piece of the code, in plain English — what it does, how it works line by line, and why it is written that way.

01	The big picture	Which file does what, and the order the browser loads them in.
02	Pressing Generate	The one function that runs the whole app, step by step.
03	Writing the prompt	How four settings become instructions a model can follow.
04	Calling the API	The actual HTTP request to Google Gemini, and its reply.
05	When things go wrong	The fallback chain and reading the reply safely.
06	The offline planner	Making a plan with no internet, and making it vary.
07	Drawing it and printing it	Building the DOM safely, and typesetting the PDF.
08	Small things that matter	Storage, animation and the draggable notes window.

PART 01

The big picture

Before any single function makes sense, it helps to know what the program is shaped like.

There are only two pages

`index.html` is the landing page that explains the project. `studio.html` is the app. They share the first two stylesheets and nothing else. There is no server, no framework and no build step — the browser opens the file and runs it.

Load order is deliberate

File	Order	Why it is in that position
<code>tokens.css</code>	1st	Defines the variables. Nothing can use a variable before it exists.
<code>base.css</code>	2nd	The reset and shared controls, built on those variables.
<code>studio.css</code>	3rd	Last, so when two rules have equal weight this one wins and the page can override the shared defaults.
<code>studio.js</code>	deferred	<code>defer</code> means "download now, run after the HTML is parsed", so the script never looks for an element that has not been created yet.

Everything is inside one wrapper

The IIFE

js/studio.js

```
(function () {  
  'use strict';  
  
  // ... the entire program lives in here ...  
  
})();
```

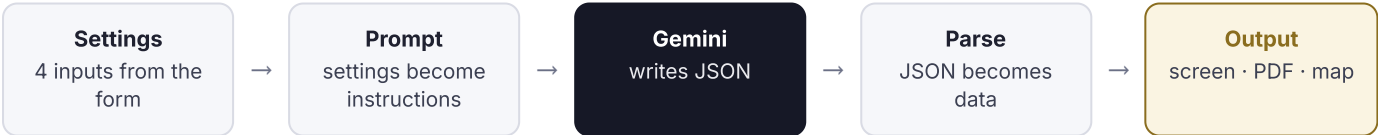
```
(function () { ... })()
```

An **IIFE** — Immediately Invoked Function Expression. The function is created and called at once. Everything declared inside it is invisible to the rest of the page, so none of my variable names can clash with anything else.

```
'use strict';
```

Turns on strict mode, which makes JavaScript report mistakes instead of quietly guessing — for example, using a variable I forgot to declare becomes an error rather than a silent global.

The path a plan takes



With no API key the middle box is replaced by the offline planner, which builds the same shape of data from a hard-coded list of real UAE places. Everything after that point is identical — which is exactly why the two modes look the same on screen, and why the app has to state which one produced the plan.

IN ONE SENTENCE

Four settings become a prompt, a model turns the prompt into JSON, and the JSON is drawn on screen, typeset into a PDF and plotted on a map.

PART 02

Pressing Generate

One function, `generate()`, drives the entire application. This is its skeleton.

The shape of the whole appjs/studio.js · generate()

```
async function generate(userMessage) {
  if (state.busy) return;           // 1
  state.busy = true;

  var cfg = readConfig();           // 2
  var key = apiKey();               // 3
  var message = userMessage || describeSettings(cfg);

  pushMessage('me', message);       // 4
  setGenerating(true, key ? 'Thinking' : 'Planning');
  var thinking = showThinking();    // 5
  setBusyPanels(true);             // 6

  if (!key) {                       // 7
    await sleep(700);
    var plan = offlinePlan(cfg, state.refinements);
    finishWithPlan(plan, cfg, notice('info', '...offline planner...'));
    done();
    return;
  }

  try {                             // 8
    var chain = await buildChain(key, model);
    var result = await callWithFallback(key, chain, body, onHop);
    var parsed = parseJSON(result.text);
    finishWithPlan(parsed, cfg, null);
    done();
  } catch (err) {                   // 9
    finishWithPlan(offlinePlan(cfg, state.refinements), cfg,
      notice('error', 'The AI request failed: ' + err.message));
    done();
  }
}
```

1 <code>if (state.busy) return;</code>	A re-entry guard . If a plan is already being built, a second click does nothing. Without this, two runs would overlap and fight over the same output panel.
2 <code>readConfig()</code>	Reads the four settings out of the form — destination, days, budget, and which interest chips are ticked — into one plain object.
3 <code>apiKey()</code>	Reads the key box. This single value decides the entire path below: no key means offline, a key means Gemini.
4 <code>pushMessage('me', ...)</code>	Shows the user's own message in the chat immediately, before any waiting starts, so the interface responds to the click straight away.
5 <code>showThinking()</code>	Adds the three bouncing dots. It carries <code>aria-label="Assistant is typing"</code> because three animated dots mean nothing to a screen reader.
6 <code>setBusyPanels(true)</code>	Raises the blurred "generating" veil over <i>both</i> the Itinerary and PDF panels — you can switch tabs mid-run, and the PDF tab must never show the previous trip as if it were the new one.
7 <code>if (!key) { ... }</code>	The offline path. The 700 ms pause is deliberate: the planner finishes in about two milliseconds, and an answer that instant feels fake.
8 <code>try { ... }</code>	The AI path: work out which models the key can call, call them, parse the reply, show the plan.
9 <code>catch (err) { ... }</code>	If every model failed, show the offline plan anyway and say what went wrong. Being told honestly and given something that works beats being left with nothing.

Why `async` and `await`? Talking to Gemini takes seconds, and JavaScript cannot simply stop and wait — that would freeze the page. `await` lets me write the code top-to-bottom as if it were waiting, while the browser stays responsive underneath. Any function containing `await` must be marked `async`.

IN ONE SENTENCE
`generate()` locks the interface, reads the settings, takes either the offline path or the Gemini path, and always finishes by showing a plan and unlocking again.

PART 03

Writing the prompt

This is the actual AI work of the project. The model is only as good as the instructions it is given.

Building the instructions

js/studio.js · buildPrompt()

```
var lines = [];
lines.push('You are an expert United Arab Emirates travel planner.');
```

```
lines.push('- Destination: ' + cfg.destination);
lines.push('- Duration: ' + cfg.days + ' days');
lines.push('- Budget per person: ' + aed(cfg.budget) + ' (' + tier.name + ')');
```

```
if (notes.length) {
  lines.push('TRAVELLER NOTES, most recent last - obey these above all:');
  notes.forEach(function (note, i) { lines.push((i + 1) + '. ' + note); });
}
```

```
lines.push('RULES');
lines.push('- Group each day geographically to minimise travel.');
```

```
lines.push('- Respect UAE realities: summer midday heat, Friday prayer timings,');
lines.push(' modest dress at mosques, Ramadan hours, drive times.');
```

```
lines.push('- 4 to 6 stops per day, each with a time window, a reason, an AED cost.');
```

```
lines.push('Reply with ONLY valid JSON. No markdown fences. Exactly this shape:');
lines.push('{\"title\":\"\", \"summary\":\"\", \"days\":[...], \"estimatedTotal\":\"AED 5,400\"}');
```

```
return lines.join('\\n');
```

'You are an expert ...'	The role . Telling a model who it is changes the whole answer. "An expert UAE travel planner" produces different writing from a general assistant.
lines.push(...)	The prompt is built as an array of lines and joined at the end. This is far easier to read and change than one enormous string with + signs everywhere.
TRAVELLER NOTES, most recent last	Everything typed in the chat is collected and sent every time. Saying <i>most recent last</i> and <i>obey these above all</i> tells the model which instruction wins when two notes conflict.
'Group each day geographically'	A constraint , and the reason the plans are usable. Without it the model happily sends you across the city and back before lunch.
'Respect UAE realities'	Local knowledge the model has but will not apply unless asked: heat, prayer times, dress codes, real drive times between emirates.
'Reply with ONLY valid JSON'	The output contract . Giving the exact shape means the reply can be fed straight into the renderer instead of being read by a human.
lines.join('\\n')	Joins everything with newlines into the single string that gets sent.

The idea to remember. An unconstrained model writes pleasant paragraphs. Pinned to a role, a currency, a pacing rule and an exact JSON shape, the same model becomes a *reliable component* that the rest of the program can depend on. That shift is what prompt engineering actually means.

One extra trick

Making Regenerate do something visible

js/studio.js

```
if (state.aiAttempts > 1) {
  lines.push('- This traveller has seen a plan for these settings already. ' +
    'Choose a noticeably different mix of stops.');
```

```
}
```

WHY

The same prompt at the same settings gets much the same answer, so Regenerate looked broken. From the second attempt onwards the prompt explicitly asks for something different.

PART 04

Calling the API

The single most important piece of code in the project: the HTTP request that reaches Google.

The request

js/studio.js · callModel()

```
var API = 'https://generativelanguage.googleapis.com/v1beta';

async function callModel(key, model, body) {
  var res = await fetch(
    API + '/models/' + encodeURIComponent(model) +
    ':generateContent?key=' + encodeURIComponent(key),
    {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(body)
    }
  );

  var data = null;
  try { data = await res.json(); } catch (err) { data = null; }

  if (!res.ok) throw httpError(res, data);

  var parts = data.candidates[0].content.parts;
  var text = parts.map(function (p) { return p.text || ''; }).join('').trim();

  if (!text) throw new Error('Empty response from ' + model);
  return text;
}
```

<code>fetch(url, options)</code>	The browser's built-in way of making an HTTP request. It returns a <i>promise</i> , which is why it needs <code>await</code> .
<code>':generateContent'</code>	The Gemini endpoint that generates text. The model name goes in the path, so a different model is a different URL.
<code>encodeURIComponent(...)</code>	Escapes anything in the model name or key that would otherwise break the URL. Never paste raw values into a URL.
<code>'?key=' + key</code>	How Gemini authenticates. Because there is no backend, the key travels from this browser straight to Google and nowhere else.
<code>method: 'POST'</code>	The prompt is data being <i>sent</i> , not a page being fetched, so it is a POST with a body.
<code>JSON.stringify(body)</code>	Turns my JavaScript object into the JSON text the API expects. Its opposite, <code>res.json()</code> , turns the reply back into an object.
<code>if (!res.ok) throw</code>	<code>fetch</code> does not throw on 404 or 500 — it resolves normally with <code>ok: false</code> . Forgetting this check is the classic <code>fetch</code> bug.
<code>data.candidates[0].content.parts</code>	Where the answer actually lives. Gemini can return several candidates, each holding parts; this takes the first candidate and joins its parts into one string.

What is sent, and what comes back

Sent

request body

```
{
  contents: [{
    role: 'user',
    parts: [{ text: thePrompt }]
  }],
  generationConfig: {
    temperature: 0.9,
    responseMimeType: 'application/json'
  }
}
```

Received

response shape

```
{
  candidates: [{
    content: {
      parts: [{ text: '{"title": ...}' }]
    }
  }]
}
```

<code>temperature: 0.9</code>	How adventurous the model is allowed to be. Near 0 it gives almost the same answer every time; 0.9 is high enough that two runs differ without becoming strange.
<code>responseMimeType</code>	Asks the API itself to enforce JSON, as a second line of defence behind the instruction in the prompt.

PART 05

When things go wrong

Most of the interesting code in this project is not the happy path — it is everything around it.

Trying model after model

js/studio.js · callWithFallback()

```
var RETRYABLE = { 404: true, 429: true, 500: true, 503: true, 504: true };

async function callWithFallback(key, chain, body, onHop) {
  var firstError = null;

  for (var i = 0; i < chain.length; i++) {
    try {
      var text = await callModel(key, chain[i], body);
      return { text: text, model: chain[i], fellBack: i > 0 };
    } catch (err) {
      if (!firstError) firstError = err;
      if (!RETRYABLE[err.status] || i === chain.length - 1) throw firstError;
      onHop(chain[i], chain[i + 1]);
      await sleep(400);
    }
  }
}
```

chain	A list of model names to try in order, already filtered so it only contains models this key can really call.
RETRYABLE	Only some failures are worth retrying: 404 (model gone), 429 (rate limited), 500/503/504 (Google having a bad moment). A 400 means <i>my</i> request was wrong, so trying another model would fail identically.
if (!firstError) firstError = err;	Keeps the first error, never the last. The first came from the model the user actually chose. Reporting the last one made a bad key show up as "gemini-1.5-pro not found" — the name of a model nobody picked.
onHop(...)	Posts a line into the chat: "gemini-X is busy — falling back to gemini-Y". A slow request should never look like a frozen one.
await sleep(400)	A short pause before retrying, so a rate limit is not hit again immediately.

The bug this was written for. The first version had model names hard-coded. Google retired them, every request came back 404, and the app silently fell back to the offline planner forever — with a perfectly valid key sitting in the box. Now it asks the API which models exist before trusting any name.

Reading a reply that is not quite JSON

js/studio.js · parseJSON()

```
function parseJSON(text) {
  var cleaned = String(text)
    .replace(/\s*````(?:json)?\s*/i, '')
    .replace(/\s*```\s*$/, '')
    .trim();

  try { return JSON.parse(cleaned); } catch (err) { }

  var first = cleaned.indexOf('{');
  var last = cleaned.lastIndexOf('}');
  if (first !== -1 && last > first) {
    try { return JSON.parse(cleaned.slice(first, last + 1)); } catch (err) { }
  }

  return null;
}
```

.replace(/\s*````.../)	Strips a markdown code fence. Models add them out of habit even when told not to, and <code>JSON.parse</code> chokes on the backticks.
try { JSON.parse } catch	First attempt on the cleaned text. If it throws, we do not give up — we fall through to attempt two.
indexOf('{') ... lastIndexOf('}')	Second attempt: take everything between the first opening brace and the last closing one. This rescues replies that begin "Sure! Here is your plan: {...".
return null;	Signals "this was not JSON". The caller then displays the model's prose as text rather than showing an error, so a usable answer is never thrown away over its formatting.

PART 06

The offline planner

The part that works with no internet and no key — and the maths that stops it repeating itself.

The problem it had to solve

The planner is a **pure function**: the same settings in, the same plan out. That is normally a good property, but it meant pressing Generate twice produced an identical plan, which reads as "the AI is broken" — and in demo mode there is no AI to blame.

A random number generator I control

js/studio.js · seeded()

```
function seeded(seed) {
  var s = (Math.imul(seed ^ 0x9e3779b9, 2654435761) >>> 0) || 1;
  return function () {
    s = (Math.imul(s, 1664525) + 1013904223) >>> 0;
    return s / 4294967296;
  };
}

function shuffled(list, rnd) {
  var a = list.slice();
  for (var i = a.length - 1; i > 0; i--) {
    var j = Math.floor(rnd() * (i + 1));
    var tmp = a[i]; a[i] = a[j]; a[j] = tmp;
  }
  return a;
}
```

seeded(seed)	Returns a <i>function</i> that produces numbers between 0 and 1 — the same job as <code>Math.random()</code> , except the sequence is decided entirely by the seed I hand it.
seed ^ 0x9e3779b9 Math.imul(...)	A hashing step. Seeds 1, 2 and 3 are very close together, and without scrambling them first the resulting sequences start out nearly identical.
s * 1664525 + 1013904223	A linear congruential generator — multiply, add, wrap around. Simple, fast, and perfectly good for shuffling a list.
>>> 0	Forces the result back into a 32-bit unsigned integer so the numbers wrap instead of growing forever.
s / 4294967296	Divides by 2 ³² to land in the 0–1 range that the shuffle expects.
shuffled(list, rnd)	A Fisher–Yates shuffle : walk the list backwards, swapping each item with a random earlier one. Every ordering is equally likely.
a = list.slice()	Copies the array first, so the original pool of places is never reordered.

Why seeded instead of `Math.random()`? One seed always gives one ordering. That keeps the planner reproducible and testable — if I want to see the same plan again, I use the same seed. A session counter feeds the seed and is saved to `localStorage`, so reloading the page does not rewind the planner to its first plan.

Reacting to the chat, honestly

Pattern matching, not understanding

js/studio.js

```
var RX_LIGHTER = /toddler|kid|child|family|slow|light|relax|tired|elderly/i;
var RX_CHEAPER = /cheap|budget|less|afford|save/i;

if (RX_LIGHTER.test(joined)) perDayStops = 3; // fewer stops per day

/...|...|.../i

.test(joined)
```

/.../i	A regular expression listing words that suggest a slower trip. The <code>i</code> makes it case-insensitive.
.test(joined)	Returns true if any of those words appear anywhere in the chat notes.

BE HONEST ABOUT THIS

This is keyword spotting, not comprehension — and the app says so in the chat. Only the Gemini path genuinely understands a note like "make day 4 lighter, we have a toddler".

PART 07

Drawing it and printing it

Turning the data back into something on screen, and then onto paper.

Building elements safely

js/studio.js · el()

```
function el(tag, className, text) {
  var node = document.createElement(tag);
  if (className) node.className = className;
  if (text !== undefined && text !== null) node.textContent = String(text);
  return node;
}
```

<code>document.createElement(tag)</code>	Creates a real element object — a <code>div</code> , a <code>span</code> — rather than writing HTML as a string.
<code>node.textContent = ...</code>	Sets the element's <i>text</i> . Anything that looks like a tag is displayed as characters, not interpreted as markup.
never <code>innerHTML</code>	This is the security point. The content comes from a language model, so it is untrusted input. If a model returned <code></code> as a stop title, <code>innerHTML</code> would run it — <code>textContent</code> just displays it.

The principle. Treat AI output exactly like something typed by a stranger: display it, never execute it. Every single node in the itinerary, the map list and the chat is built with this helper.

Typesetting the PDF

js/studio.js · buildPDF()

```
var doc = new window.jspdf.jsPDF({ unit: 'pt', format: 'a4' });

doc.setFillColor(22, 24, 38);
doc.rect(0, 0, pageW, 116, 'F');
doc.setTextColor(145, 132, 217);
doc.text(plan.title, margin, 94);

function need(height) {
  if (y + height ≤ pageH - 56) return;
  doc.addPage();
  y = margin + 18;
}

if (state.pdfUrl) URL.revokeObjectURL(state.pdfUrl);
state.pdfUrl = URL.createObjectURL(doc.output('blob'));
pdfFrame.src = state.pdfUrl + '#view=FitH';
```

<code>unit: 'pt', format: 'a4'</code>	Work in points on an A4 page — the units a printed document is actually measured in.
<code>doc.rect(..., 'F')</code>	Draws the navy header band. 'F' means fill it rather than outline it.
<code>doc.text(str, x, y)</code>	Writes real, selectable text at a position. This is why the output can be searched and copied — it is not a screenshot.
<code>need(height)</code>	My own page-break helper: before drawing a block, check it fits; if not, start a new page. This stops a day heading landing alone at the foot of a page.
<code>createObjectURL(blob)</code>	Turns the finished document into a temporary URL the iframe can display, so you see exactly what will download.
<code>revokeObjectURL(...)</code>	Releases the <i>previous</i> URL first. Without this, every regeneration leaks a whole PDF into memory for as long as the tab is open.

PART 08

Small things that matter

Three short pieces of code that are easy to miss but worth being able to explain.

Storage that cannot break the page

Every read and write is wrapped

js/studio.js

```
function save(name, value) {  
  try { window.localStorage.setItem(name, value); } catch (err) { }  
}
```

```
try { ... } catch (err) { }
```

`localStorage` **throws** in private browsing and under `file://` in some browsers. Nothing stored here — a remembered key, some notes — is important enough to break the whole page over, so failure is simply ignored.

The typing animation

Revealing text a few characters per frame

js/studio.js · typeOut()

```
var cpf = Math.min(9, Math.max(1, Math.ceil(text.length / frames)));  
  
(function step() {  
  if (token.cancelled) { node.textContent = text; resolve(); return; }  
  var stick = chatScrolledToBottom();  
  shown += cpf;  
  node.textContent = text.slice(0, shown);  
  if (stick) dom.chatLog.scrollTop = dom.chatLog.scrollHeight;  
  if (shown < text.length) window.requestAnimationFrame(step);  
})();
```

```
requestAnimationFrame
```

Runs the next step just before the screen repaints, so the animation stays smooth — and it *pauses automatically* in a background tab, which `setInterval` would not.

```
Math.min(9,  
Math.max(1, ...))
```

Clamps the speed between 1 and 9 characters a frame, so a short reply still feels typed and a long one does not outstay its welcome.

```
token.cancelled
```

A cancellation flag. Starting a second generation cancels the first, so two messages can never type themselves at once.

```
if (stick) ... scrollTop
```

Only auto-scrolls if the user was already at the bottom. Yanking the view back while someone is reading an earlier message is the fastest way to make a chat feel hostile.

Dragging the notes window

Pointer events with capture

js/studio.js · initNotes()

```
bar.addEventListener('pointerdown', function (event) {  
  dragging = true;  
  var rect = notes.getBoundingClientRect();  
  offsetX = event.clientX - rect.left;  
  offsetY = event.clientY - rect.top;  
  bar.setPointerCapture(event.pointerId);  
});  
  
var x = Math.min(Math.max(0, event.clientX - offsetX), maxX);
```

```
pointerdown
```

Pointer events cover mouse, touch and pen with one set of handlers, instead of writing mouse and touch versions separately.

```
setPointerCapture
```

Keeps sending events to the title bar even when the cursor moves off it — and it will, because people drag quickly.

```
event.clientX  
- rect.left
```

Remembers *where inside the bar* you grabbed it, so the window does not jump so its corner snaps to the cursor.

```
Math.min(Math.max(...))
```

Clamps the window inside the viewport so it can never be dragged somewhere unreachable.

IF YOU ARE ASKED WHAT YOU ARE MOST PROUD OF

Not the features — the failure handling. The app keeps working with no internet, no key, a retired model, or a reply that is not valid JSON, and it tells you honestly which of those happened.